LIBRARY MANAGEMENT SYSTEM

Project Documentation

Java Desktop Application with MySQL Database

Project Title	Library Management System
Technology	Java, Swing, JDBC, MySQL
Development Environment	Eclipse IDE / JDK / MySQL Workbench
Student Name	K hemadri
Roll Number	23BFA05197
Institution	Sri Venkateswara College of Engineering
Academic Year	2026

Prepared for project / internship submission

DOCUMENT OVERVIEW

This documentation was prepared from the uploaded **LibraryManagementSystem** source project. It describes the implemented features, application structure, database design, processing flow, testing approach, limitations, and possible future improvements. The document intentionally avoids including the database password contained in the source code.

Item	Details
Application type	Desktop-based Library Management System
Programming language	Java
GUI technology	Java Swing
Database	MySQL
Database access	JDBC using PreparedStatement and ResultSet
Database name used by the project	library_management
Main package	library
Primary entities	Books, Students, Issued Books
Interfaces implemented	Console menu and Swing graphical interface

TABLE OF CONTENTS

1. ABSTRACT

The Library Management System is a Java-based desktop application designed to simplify common library operations such as maintaining book records, maintaining student records, searching for books, issuing books, returning books, and monitoring issued-book information. The project uses Java Swing for the graphical user interface and JDBC for communication with a MySQL database. The database stores persistent information about books, students, and book transactions.

The implementation contains both a console-driven **Main** class and a Swing-based **libraryGUI** class. The GUI provides forms and tables for day-to-day operations, while the console implementation demonstrates the same core database operations through a menu.

A key feature is quantity tracking. Each book record stores both total quantity and available quantity. When a book is issued, available quantity is reduced by one; when it is returned, available quantity is increased by one. This provides a simple mechanism for tracking circulation.

2. INTRODUCTION

Manual library record keeping can become difficult when the number of books and students increases. Searching paper records, updating quantities, and tracking issued books can take time and can lead to inconsistent records. The purpose of this project is to provide a basic computerized solution for these operations.

The project follows a database-driven desktop application approach. Java handles application logic and user interaction, JDBC provides the database connectivity layer, and MySQL stores the records. The uploaded source contains separate Java classes for individual operations, which keeps the basic functions easy to understand and test.

3. PROBLEM STATEMENT

A library needs a reliable method to maintain book information, student information, and issue/return transactions. The system should allow an operator to add and update records, quickly find a book, issue a book only when it is available, return an issued book, and view current records.

The project addresses these requirements by providing:

- Centralized storage of library records in MySQL.
- CRUD operations for book and student records.
- Title-based book search using SQL LIKE.
- Issue validation based on student existence and book availability.
- Automatic available-quantity updates during issue and return.
- A graphical interface using Java Swing tables, forms, buttons, and dialogs.

4. OBJECTIVES

- Develop a functional library management application using Java.
- Connect the Java application to MySQL using JDBC.
- Implement create, read, update, and delete operations for books and students.
- Implement book issue and return workflows.
- Maintain total and available book quantities.
- Provide a simple graphical interface for library operations.
- Demonstrate practical use of PreparedStatement, ResultSet, executeQuery(), and executeUpdate().

• Organize application functionality into separate Java classes.

5. SCOPE OF THE PROJECT

Included in the current implementation:

• Book management: add, view, search, update, and delete.

• Student management: add, view, update, and delete.

• Book issuing after checking student existence and book availability.

• Book return after checking for an active issue.

• Viewing issued-book records.

• Persistent storage using MySQL.

• Console and Swing-based interaction.

Not included in the current implementation:

• User login, role-based access, or administrator authentication.

• Fine calculation and payment handling.

• Email/SMS notifications.

• Cloud or web deployment.

• Book reservation or waiting-list management.

• Automated backup and recovery.

6. TECHNOLOGIES AND TOOLS

Technology / Tool	Purpose
Java	Application programming and business logic
Java Swing	Graphical user interface
JDBC	Communication between Java and MySQL
MySQL	Persistent relational database
PreparedStatement	Parameterized SQL execution
ResultSet	Reading records returned by SELECT queries
Eclipse IDE	Java project development environment
MySQL Workbench	Database creation and inspection

7. SYSTEM ARCHITECTURE

The application uses a simple three-part logical structure. The user interacts with either the Swing GUI or the console menu. Operation classes execute the required SQL statements through the DBConnection class. MySQL stores the persistent data.

User	Presentation Layer	Application / Data Access	Database
Library operator	libraryGUI / Main	book, students, searchbook, issuebook, returnbook, update/delete/view classes + DBConnection	MySQL: library_management

High-level data flow:

• User enters data or selects an operation.

• Java validates basic input such as required fields and numeric values.

• The relevant operation class opens a JDBC connection through DBConnection.

• A parameterized SQL statement is prepared and executed.

• The result is displayed in the console or Swing table/dialog.

• For issue/return operations, the transaction record and available quantity are updated.

8. FUNCTIONAL MODULES

Module	Main operations	Relevant classes
Book Management	Add, view, search, update, delete	book, viewbooks, searchbook, updatebook, deletebook
Student Management	Add, view, update, delete	addstudents, viewstudents, updatestudent, deletestudent
Issue Management	Check student, check book, issue book, reduce availability	issuebook
Return Management	Check active issue, set return date, increase availability	returnbook
Issued Books	View issue history	viewissuedbooks
Database Connectivity	Open MySQL JDBC connection	DBConnection
Application Entry	Console menu / GUI startup	Main, libraryGUI

9. DATABASE DESIGN

The SQL statements in the uploaded source show three primary tables: **books**, **students**, and **issued_books**. The following structure is documented from the columns referenced by the Java source.

9.1 Books Table

books column	Type / role	Description
id	Integer, identifier	Book identifier used by issue/return operations
title	Text	Book title
author	Text	Author name
category	Text	Book category
publisher	Text	Publisher
quantity	Integer	Total number of copies
available_quantity	Integer	Copies currently available

9.2 Students Table

students column	Type / role	Description
no	Integer, identifier	Student record number; GUI reads this field
name	Text	Student name
roll_no	Integer, unique identifier	Student roll number used by operations
department	Text	Student department

students column	Type / role	Description
email	Text	Student email address

9.3 Issued Books Table

issued_books column	Type / role	Description
issue_id	Integer, identifier	Issue transaction identifier
roll_no	Integer	Student associated with the issue
book_id	Integer	Book associated with the issue
book_title	Text	Stored title of issued book
issue_date	Date	Date on which the book was issued
return_date	Date / NULL	Return date; NULL indicates an active issue

Relationship concept: a student can have issue records, and a book can appear in issue records. The current Java code uses roll_no and book_id to identify these relationships in SQL. The uploaded project does not include a schema/SQL file, so exact foreign-key constraints cannot be confirmed from the archive alone.

10. DETAILED MODULE DESCRIPTION

10.1 Book Management

- **Add Book:** accepts title, author, category, publisher, and quantity. It first checks whether a matching book already exists. If it exists, quantity and available_quantity are increased; otherwise a new record is inserted.
- **View Books:** retrieves records using SELECT * FROM books and displays book information.
- **Search Book:** uses a parameterized LIKE query so a partial title can be searched.
- **Update Book:** allows title, author, category, publisher, or quantity to be changed.
- **Delete Book:** deletes a book record by ID.

10.2 Student Management

- **Add Student:** stores name, roll number, department, and email.
- **View Students:** retrieves student records from MySQL.
- **Update Student:** updates name, department, and email based on roll number.
- **Delete Student:** removes a student record using roll number.

10.3 Issue Book

- Checks whether the supplied student roll number exists.
- Checks whether the requested book title exists.
- Checks available_quantity before issuing.
- Inserts a new issued_books record with CURDATE() as issue_date and NULL as return_date.
- Decreases books.available_quantity by one after successful insertion.

10.4 Return Book

- Checks for an active issue where return_date IS NULL.
- Updates return_date to CURDATE().
- Increases books.available_quantity by one.

- Shows a success or error message depending on the operation.

10.5 View Issued Books

The Swing implementation loads issue_id, roll number, book ID, book title, issue date, and return date into a JTable. A NULL return date represents a book that has not yet been returned.

11. IMPORTANT PROCESSING LOGIC

11.1 Book availability

The central quantity rule is:

available_quantity = number of copies currently available for issue
When issuing: available_quantity = available_quantity – 1
When returning: available_quantity = available_quantity + 1

11.2 Search logic

Book search uses a parameterized SQL statement equivalent to `SELECT * FROM books WHERE title LIKE ?`, with the entered title surrounded by percent signs. This supports partial title matching.

11.3 JDBC execution

- SELECT statements use executeQuery() and return a ResultSet.
- INSERT, UPDATE, and DELETE statements use executeUpdate().
- PreparedStatement parameters are set using methods such as setString() and setInt().
- Database connections are obtained from DBConnection.getConnection().

12. USER INTERFACE

The Swing interface in libraryGUI.java creates separate windows for major operations. The project uses standard Swing components including JFrame, JLabel, JTextField, JButton, JTable, JScrollPane, DefaultTableModel, and JOptionPane.

Window / action	Interface elements	Expected result
Add Book	Text fields + quantity + button	Book inserted or existing quantity increased
View Books	JTable	All book records displayed
Search Book	Title field + Search + JTable	Matching books displayed
Update Book	ID + fields + Update	Selected book fields updated
Add/View/Update/Delete Student	Forms, tables, confirmation dialog	Student records managed
Issue Book	Roll No + Book Title	Issue record created and availability reduced
Return Book	Roll No + Book ID	Return date stored and availability increased
View Issued Books	JTable	Issue history displayed

13. TESTING AND VALIDATION

The following test cases are derived from the implemented validations and database operations. They can be used as the functional testing record for the project.

TC	Test case	Expected result
01	Add a new book with valid data	Book record is inserted successfully
02	Add an already existing book	Quantity and available quantity are increased
03	Search using part of a title	Matching title records are displayed
04	Search for a nonexistent title	Book-not-found message is shown
05	Update an existing book	Selected field is updated
06	Delete an existing book	Book record is deleted
07	Add student with valid roll number	Student is inserted
08	Update student details	Name/department/email are updated
09	Issue available book to existing student	Issue record created; available quantity decreases
10	Issue unavailable book	Issue is rejected
11	Issue using nonexistent student roll number	Issue is rejected
12	Return an actively issued book	Return date stored; available quantity increases
13	Return a book without an active issue	Return is rejected
14	View issued books	Issue records are displayed

Observed validation mechanisms in source code:

- Empty-field checks in the Swing forms.
- NumberFormatException handling for numeric fields.
- Existence checks before student update/delete and book issue.
- Availability check before issuing a book.
- Active-issue check before returning a book.
- Success/failure messages using console output or JOptionPane.

14. SECURITY AND DATA INTEGRITY CONSIDERATIONS

The application uses PreparedStatement for its SQL parameters, which is preferable to directly concatenating user input into SQL statements. This reduces SQL-injection risk for the parameterized queries implemented in the project.

However, the uploaded DBConnection.java contains a database username and password directly in the source code. The password is intentionally not reproduced in this documentation. For a real deployment, credentials should be moved to environment variables, a protected configuration file, or a secrets management mechanism.

For stronger transaction integrity, the issue insertion and available-quantity update should ideally be executed within one database transaction. Otherwise, a failure between the two statements could leave the issue record and quantity temporarily inconsistent.

15. LIMITATIONS

- No authentication or role management is implemented.
- The project is designed for a local MySQL database and desktop execution.
- There is no fine/late-return calculation.
- No transaction rollback is implemented around the issue/return multi-step operations.

- Some console and GUI implementations use slightly different input fields; for example, console return uses book title while the GUI return uses book ID.
- The uploaded archive contains source and compiled classes but no database schema/SQL export, so exact database constraints cannot be verified from the archive.
- Some console display methods are minimal and are intended mainly for demonstration.

16. FUTURE ENHANCEMENTS

- Add secure login with Admin/Librarian roles.
- Use foreign keys between students, books, and issued_books.
- Wrap issue and return operations in JDBC transactions with commit/rollback.
- Add due dates, overdue detection, and automatic fine calculation.
- Add book reservation and availability notifications.
- Add dashboard statistics such as total books, available books, issued books, and registered students.
- Improve GUI layout using a modern layout manager instead of absolute positioning.
- Add validation for email format, positive quantity, duplicate roll numbers, and invalid IDs.
- Move database credentials out of source code.
- Add automated unit/integration tests and database backup functionality.
- Optionally migrate the project to a web application using Spring Boot for multi-user access.

17. CONCLUSION

The Library Management System demonstrates how Java, Swing, JDBC, and MySQL can be combined to build a practical database-driven desktop application. The project covers the fundamental operations required for a small library: managing books and students, searching records, issuing and returning books, and tracking availability.

The source code also demonstrates important Java database programming concepts such as JDBC connections, PreparedStatement, ResultSet, executeQuery(), executeUpdate(), exception handling, and event-driven Swing programming. With the recommended enhancements—especially authentication, stronger database constraints, transaction handling, and improved validation—the project can be developed into a more robust library management solution.

18. REFERENCES

- Java SE documentation and standard library concepts.
- Java Swing components and event handling concepts.
- JDBC concepts for relational database connectivity.
- MySQL SQL syntax for SELECT, INSERT, UPDATE, DELETE, and LIKE.
- Project source code supplied in the uploaded LibraryManagementSystem archive.

APPENDIX A — SOURCE PROJECT STRUCTURE

File	Purpose
Main.java	Console menu and application entry point
libraryGUI.java	Swing graphical user interface
DBConnection.java	MySQL JDBC connection

File	Purpose
book.java	Add book and duplicate-book quantity handling
viewbooks.java	Console book listing
searchbook.java	Console title search
updatebook.java	Console book update
deletebook.java	Console book deletion
addstudents.java	Console student insertion
viewstudents.java	Console student listing
updatestudent.java	Console student update
deletestudent.java	Console student deletion
issuebook.java	Book issue operation
returnbook.java	Book return operation
viewissuedbooks.java	Console issued-book listing

APPENDIX B — SAMPLE SQL OPERATIONS

```
SELECT * FROM books;

SELECT * FROM books WHERE title LIKE ?;

INSERT INTO students(name, roll_no, department, email) VALUES (?, ?, ?, ?);

UPDATE issued_books SET return_date=CURDATE() WHERE roll_no=? AND book_id=? AND return_date IS NULL;

UPDATE books SET available_quantity=available_quantity-1 WHERE id=?;

UPDATE books SET available_quantity=available_quantity+1 WHERE id=?;
```